

As an AI analyzing the underlying architecture of your **MASTER_LIST_MERGE** framework, I can see exactly why the V8 engine is choking on these specific steps. The framework relies heavily on sequential **SpreadsheetApp** calls and massive in-memory data structures.

Here are my technical recommendations to fundamentally improve the speed of your critical bottlenecks and slow processes.

1. Resolve the CRITICAL In-Memory Bottleneck (Monthly Change Report)

The Problem: The **compareRawDemoPForSectionReport_** function is pulling over 12,000 rows (current and previous months) into memory at the exact same time. It then loops through these massive arrays to build JavaScript **Map** and **Set** objects for comparison. This maxes out the Google Apps Script 50MB memory limit and CPU allocation, causing the 82-second to 168-second stall. **The Solution:**

- **Offload to Native Sheets Backend:** Instead of using JavaScript to compare 50+ columns cell-by-cell, offload this to the Google Sheets backend. You can temporarily write the two datasets to a hidden sheet and use native **QUERY** or **FILTER** formulas to identify the deltas instantly.
- **Use Hash Comparisons:** If it must remain in JS, compute a single string hash (e.g., MD5) for each row upon ingestion. Compare the single hash string rather than iterating through all 50 columns for every single participant.

2. Bypass Sequential API Overhead (Sheet Insertion & Template Copying)

The Problem: Steps like "blank sheet inserted in governed order" or "copy template for archive" are taking between 10 and 40+ seconds. The **SpreadsheetApp** service makes a separate HTTP request for every structural action (insert sheet, set tab color, freeze rows, copy formats). **The Solution:**

- **Migrate to the Advanced Sheets API:** Enable the **Sheets** advanced service in your Apps Script project. You can bundle sheet creation, naming, template copying, and formatting into a single **Sheets.Spreadsheets.batchUpdate** JSON payload. This executes on Google's servers in one network call, reducing 40-second operations to less than a second.

3. Optimize the Index Refresh (Disenrolled List & Imports)

The Problem: The **createIndexSheet()** function takes around 30 seconds. Every time a macro finishes, it deletes the index, reads every sheet in the workbook, generates dynamic **HYPERLINK** formulas, and rebuilds the visual matrix (including alternating colors) entirely from scratch. **The Solution:**

- **Targeted Updates:** Stop dropping and rebuilding the entire index. Instead, map the existing index into memory, find the specific row that needs to be updated (e.g., changing a sheet from "Visible" to "Hidden"), and use `setValues()` to update *only* that row.
- **Batch Backgrounds:** If you must rebuild it, apply the alternating colors using Google Sheets' native Banding feature (`range.applyRowBanding()`) rather than pushing massive 2D arrays of hex codes via `setBackgrounds()`.

4. Streamline Data Appends (Disenrolled List)

The Problem: Appending payloads to the Disenrolled List takes ~26 seconds for 700 rows. This happens because formatting is being copied and applied dynamically as new rows are injected.

The Solution:

- **Pre-Format the Target Range:** Insert blank rows, apply the formatting via the Advanced API (or `copyTo` with `PASTE_FORMAT`) *before* pushing the values, and then run your `setValues()` array.
- **Suspend Calculations:** Large copy-paste operations trigger spreadsheet recalculations. Grouping your data mutations tightly together limits the number of times the UI attempts to redraw the sheet.

Which of these architectural shifts—handling the heavy memory operations, refactoring to the Advanced Sheets API, or optimizing the Index rebuild—would you like to explore first?

Codex's recommendations are exceptionally well-diagnosed. They correctly identify the symptoms and provide an excellent roadmap for adding granular telemetry.

However, Codex focuses heavily on *diagnostics* (adding more timing steps) and logical restructuring. To actually hit Codex's strict acceptance targets (e.g., getting the Monthly Change dataset compilation below 15 seconds), you need to combine their logical roadmap with Google Apps Script (GAS) specific hardware workarounds.

Here is my additional input on *how* to execute Codex's recommendations at the codebase level:

1. On Priority 1 — Monthly Change Dataset Compilation

- **Implement MD5 Row Hashing:** Codex recommends building "normalized row metadata." Take this a step further: when `getRawDemoPDataForCompare_` ingests the arrays, use `Utilities.computeDigest(Utilities.DigestAlgorithm.MD5, ...)` to generate a single hash string for the tracked demographic/caseload columns. Instead of looping through `columnsToCompare.forEach` (which triggers thousands of array lookups), you simply compare `currentHash === previousHash`.
- **V8 Engine Optimization:** The current script uses nested `.forEach()` loops and `Array.from().sort()` heavily inside `getChangedColumnsForSectionRows_`. Refactor this to use native JS `Set` intersections and `Map` lookups, which the V8 engine handles exponentially faster than array iteration.

2. On Priority 2 — Disenrolled Append and Index Refresh

- **The "Top-Insertion" Penalty:** Codex hypothesizes the bottleneck is row insertion and suggests testing "append-to-bottom." **Codex is 100% correct.** Currently, `appendDisenrolledDeltasToExclusionSheet_` uses `exclusionSheet.insertRowsBefore(insertStartRow, appendPayload.length)`. Inserting rows at the top of a spreadsheet forces Google's backend to recalculate and shift every single subsequent row and format down. You must switch this to append the rows to `sheet.getLastRow() + 1`, and *then* run your governed sort. This alone will likely shave 10-15 seconds off the process.
- **Batch Format Copying:** The script currently copies formatting after inserting rows. Ensure that `SpreadsheetApp.CopyPasteType.PASTE_FORMAT` is applied to the entire block at once, rather than row-by-row or chunk-by-chunk.

3. On Priority 3 — Format Monthly Sheets Fast Canvas

- **Bypass the Standard API:** Codex notes that inserting a blank sheet took 40.562 seconds and suggests testing a "lightweight blank canvas template". Even copying a lightweight

template via `sheet.copyTo()` relies on the standard `SpreadsheetApp` service, which is subject to severe queue latency on Google's end.

- **Use the Advanced Sheets API:** Rewrite the `createOutputSheetFromDashboardTemplate_` function to use the **Advanced Sheets API** (`Sheets.Spreadsheets.batchUpdate`). You can construct a single JSON payload that inserts the sheet, names it, sizes the grid, applies the headers, and sets the frozen rows in *one single HTTP request*. This bypasses the Apps Script synchronous queue and will guarantee your sub-10-second target.

4. On Priority 4 — Hide Monthly Import Sheets

- **The Hidden Sort Cost:** Codex mentions the "Complete" step is too broad. The reason this step is bottlenecking is hiding inside `hideMonthlySheetsBySpecs_`: it unconditionally calls `enforceGlobalSheetSortOrder_(parsedMonth.monthParts)`.
- **Short-Circuit the Sort:** `enforceGlobalSheetSortOrder_` pulls all sheets, calculates their rank, and potentially triggers `ss.moveActiveSheet()`. Add a check: if `result.hidden.length === 0` (meaning no sheets actually changed visibility state), completely skip the global sort and index refresh.

5. On Priority 5 — Dashboard Quality H-N Writes

- **Write Coalescing:** Codex recommends skipping writes when the payload signature is unchanged. The codebase already attempts this via `tryDashboardQualityAnchoredColumnWrite_`, but it often falls back to `.clearContent()` and `.setValues()`. If you transition the Dashboard Quality report to use `applyRowBanding()` for its alternating colors instead of painting 2D hex-code arrays via `setBackgrounds()`, the write payload drops significantly in size, speeding up the H-N batch update.

Codex is absolutely right that v1.6.76 should not be cleared for performance until these are addressed.

Would you like me to draft the specific refactored code for the "append-to-bottom" Disenrolled

To wire the Advanced Sheets API into your heaviest output reports, we need to replace the slow, sequential `SpreadsheetApp` formatting steps (resizing grids, copying formats row-by-row, setting frozen rows, copying tab colors) with a single, lightning-fast JSON payload.

Here is the implementation.

Step 1: Add the Universal Advanced API Fast Canvas Engine

Add this new helper function anywhere in your framework (e.g., near your other template functions). This acts as the universal engine: it duplicates the template to the correct governed position, right-sizes the grid, and wipes the data zone clean—**all in one sub-second server-side request**.

JavaScript

None

```
function buildFastCanvasAdvanced_(ss, template, newSheetName,
requiredRows, requiredCols) {

    const newSheetId = Math.floor(Math.random() * 2147483647);

    const targetIndex = getGovernedSheetInsertIndex_(ss,
newSheetName);

    const batchRequests = [

        {

            duplicateSheet: {

                sourceSheetId: template.getSheetId(),

                insertSheetIndex: Math.max(targetIndex - 1, 0),

                newSheetId: newSheetId,

                newSheetName: newSheetName

            }

        },

        {

            updateSheetProperties: {

                properties: {

                    sheetId: newSheetId,
```

```

        gridProperties: {
            rowCount: requiredRows,
            columnCount: requiredCols
        }
    },
    fields: "gridProperties(rowCount,columnCount)"
}

{
    updateCells: {
        range: {
            sheetId: newSheetId,
            startRowIndex: DATA_START_ROW - 1,
            endRowIndex: requiredRows,
            startColumnIndex: 0,
            endColumnIndex: requiredCols
        },
        fields: "userEnteredValue" // Clears residual template
data, keeps exact formatting
    }
}

];

```

```

    try {

        Sheets.Spreadsheets.batchUpdate({ requests: batchRequests },
ss.getId());

        return ss.getSheetByName(newSheetName);

    } catch (err) {

        throw new Error("Advanced API canvas generation failed: " +
err.message);

    }

}

```

Step 2: Refactor `createActiveDemoPFromRawData_`

Replace your entire `createActiveDemoPFromRawData_` function with this updated version. Notice how dozens of `template.getRange(...).copyTo(...)` and `setFrozenRows` calls have been completely eliminated.

JavaScript

None

```

function createActiveDemoPFromRawData_(rawSheet, targetName,
monthParts, timing) {

    const preflight = validateRawDataPreflightForDemoP_(rawSheet,
null, "Build Demo P from Raw Data");

    const rawData = preflight.rawData;

    const movedResult = { pmrsRemoved: 0, rowsRemoved: 0, rowsCopied:
0 };

```

```
    setLastRawDataDisenrolledBuildResult_(movedResult);

    if (timing) markFrameworkStep_(timing, "Raw Data rows prepared  
for Demo P without disenrollment exclusion");

    const dashboard = loadDashboardConfig_();

    const sheetDef = getSheetDefinitionByType_(dashboard,  
SHEET_TYPE.DEMO_P);

    const headers = getHeadersForSheetType_(dashboard,  
SHEET_TYPE.DEMO_P);

    const outputRows = mapRowsToHeaders_(rawData.values,  
rawData.headers, headers);

    const workingData = {
        headers: headers,
        headerMap: buildHeaderIndexMap_(headers),
        values: outputRows,
        range: null
    };

    populateParticipantNameData_(workingData, null);
    populateDemoPNameData_(workingData, null);
    updateBannerColumnData_(workingData, null);
    combineAddressesData_(workingData, null);
```



```
handleLanguageData_(workingData, null);

splitPhoneNumbersData_(workingData, null);

runMasterContactProcessData_(workingData, null);

combineNotesSummaryData_(workingData, null);


    populateDemoPUpdateColumns_(workingData, monthParts,
rawSheet.getName(), "Created");

    populateUniversalMetadataColumns_(workingData, monthParts,
rawSheet.getName(), "Demo P", "Created");

    const flatCount = flattenDemoPContactRowsInMemory_(workingData,
true);


    if (timing) markFrameworkStep_(timing, "Demo P Data processed,
metadata mapped, and contacts flattened in memory | Rows retained:
" + flatCount);


    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const template = ss.getSheetByName(sheetDef.templateName);

    if (!template) throw new Error("Template not found: " +
sheetDef.templateName);


    deleteSheetIfExists_(ss, targetName, rawSheet.getName(),
sheetDef.templateName);
```

```
    const width = Math.max(headers.length, 1);

    const requiredRows = workingData.values.length ? DATA_START_ROW +
workingData.values.length - 1 : DATA_START_ROW;

    // --- ADVANCED API FAST CANVAS ---

    const demoSheet = buildFastCanvasAdvanced_(ss, template,
targetName, requiredRows, width);

    demoSheet.showSheet();

    ss.setActiveSheet(demoSheet);

    if (timing) markFrameworkStep_(timing, "Advanced API structural
canvas built and formatted");

    applyGovernedTextAndNumberFormats_(demoSheet, dashboard, headers,
DATA_START_ROW, workingData.values.length);

    if (workingData.values.length) {

        demoSheet.getRange(DATA_START_ROW, 1,
workingData.values.length,
width).setValues(workingData.values.map(function(row) {

            const out = (row || []).slice(0, width);

            while (out.length < width) out.push("");

            return out;

        })));
    }
}
```

```
    if (timing) markFrameworkStep_(timing, "Demo P unified values
flushed to spreadsheet canvas");

    try {

        demoSheet.getRange("A1").setValue(sheetDef.reportTitle || "Demo
P");

        const existingFilter = demoSheet.getFilter &&
demoSheet.getFilter();

        if (existingFilter) existingFilter.remove();

        demoSheet.getRange(HEADER_ROW, 1,
Math.max(demoSheet.getMaxRows() - HEADER_ROW + 1, 1),
width).createFilter();

    } catch (err) {

        logBestEffortWarning_("Demo P metadata update skipped: " +
err.message);

    }

    applyUniversalFastCanvasFormatting_(demoSheet, dashboard,
headers);

    clearSheetRuntimeCachesForSheet_(demoSheet);

    return {

        sheet: demoSheet,

        movedResult: movedResult,
```

```

    createdFromDashboardTemplate: true,

    sourceWasRawData: true,

    preProcessedWorkingSource: true,

    flatCount: flatCount
  };
}

```

Step 3: Refactor `createMasterListSheetFromTemplate_`

Replace your existing Master List builder function with this to instantly eliminate the formatting bottleneck on your final output.

JavaScript

None

```

function createMasterListSheetFromTemplate_(ss, targetName,
monthParts, timing, timingLabel) {

  ss = ss || SpreadsheetApp.getActiveSpreadsheet();

  const prefix = String(timingLabel || "Master List canvas
detail");

  const markCanvasStep = function(label, details) {

    if (timing) markFrameworkStep_(timing, prefix + " - " + label +
": " + targetName, details || "");

  };

  const existing = ss.getSheetByName(targetName);

```

```
markCanvasStep("existing sheet lookup complete", existing ?
"Existing sheet found" : "No existing sheet found");

if (existing && ss.getSheets().length > 1) {

    ss.deleteSheet(existing);

    markCanvasStep("existing sheet deleted");

}

const dashboard = loadDashboardConfig_();

markCanvasStep("dashboard configuration loaded");

const sheetDef = getSheetDefinitionByType_(dashboard,
SHEET_TYPE.MASTER_LIST);

const headers = getHeadersForSheetType_(dashboard,
SHEET_TYPE.MASTER_LIST);

markCanvasStep("headers resolved | Columns: " + headers.length);

const template = ss.getSheetByName(sheetDef.templateName);

markCanvasStep("template lookup complete",
sheetDef.templateName);

if (!template) {

    throw new Error("Template not found: " + sheetDef.templateName
+ ". Run Create / Refresh Master List Template first.");

}
```

```

    const width = Math.max(headers.length, 1);

    const requiredRows = Math.max(template.getMaxRows(),
DATA_START_ROW);

    // --- ADVANCED API FAST CANVAS ---

    const sheet = buildFastCanvasAdvanced_(ss, template, targetName,
requiredRows, width);

    markCanvasStep("Advanced API Fast Canvas built", "Rows: " +
requiredRows + "; Columns: " + width);

    showSheetIfNeeded_(sheet, timing, prefix + " - sheet visibility
enforced visible: " + targetName);

    applyColumnWidths_(sheet, dashboard, headers);

    markCanvasStep("column widths applied");

    applyGovernedTextAndNumberFormats_(sheet, dashboard, headers,
DATA_START_ROW, requiredRows >= DATA_START_ROW ? requiredRows -
DATA_START_ROW + 1 : 0);

    markCanvasStep("governed text/date/number formats applied");

    try {

        sheet.getRange("A1:D2").setValues([

            ["Master List", "", "", ""],

            ["Date:", monthParts.firstDay, "to", monthParts.lastDay]

        ]);

```

```

        markCanvasStep("title/date values stamped");

        if (headers.length) sheet.getRange(HEADER_ROW, 1, 1,
width).setValues([headers]);

        markCanvasStep("header values stamped");

    } catch (err) {

        logBestEffortWarning_("Master List metadata initialization
skipped: " + err.message);

    }

    clearSheetRuntimeCachesForSheet_(sheet);

    markCanvasStep("runtime caches cleared");

    return sheet;

}

```

Here are the refactored Banner Formatter functions. I have updated both the standalone menu function (`formatBannerReport`) and the bulk processor function (`formatMonthlyBannerSheet_`) so that both workflows benefit from the Advanced API speed boost.

By calculating the `requiredRows` ahead of time based on the Raw Data size, we completely bypass the slow `ensureSheetHasAtLeastRows_` dynamic resizing that was bottlenecking the old script.

Banner Formatter Advanced API Refactor

Replace your existing `formatBannerReport` and `formatMonthlyBannerSheet_` functions with these:

JavaScript

None

```
function formatBannerReport() {  
    return runFrameworkTimed_("Format Banner Report",  
function(timing) {  
        const ss = SpreadsheetApp.getActiveSpreadsheet();  
  
        const dashboard = loadDashboardConfig_();  
  
        const sheetDef = getSheetDefinitionByType_(dashboard,  
SHEET_TYPE.BANNER);  
  
        const rawSheetOriginal = ss.getActiveSheet();  
  
        if (!rawSheetOriginal) {  
            throw new Error("No active sheet found. Open/select the raw  
Banner import sheet before running Format Banner Report.");  
        }  
  
        const monthParts = promptForLockedYearReportMonth_("Format  
Banner Report");  
  
        if (!monthParts) return null;  
  
        const reportDate = monthParts.firstDay;  
  
        const rawName = "Raw Data - Banners " +  
Utilities.formatDate(reportDate, Session.getScriptTimeZone(),  
"MM.yy");  
  
        const outputName = buildBannerReportOutputName_(reportDate);  
  
        const templateName = "Template - Banner Report";  
    }  
}
```



```
    if (rawSheetOriginal.getName() === templateName) {

        throw new Error("The active sheet is the Banner template.  
Select the raw Banner import sheet before running the formatter.");

    }

    if (rawSheetOriginal.getName() === RFF_TIMING_SHEET ||  
rawSheetOriginal.getName() === RFF_DASHBOARD_SHEET) {

        throw new Error("The active sheet is not a raw Banner import  
sheet. Select the raw Banner import sheet before running the  
formatter.");

    }

    const rawSheet = rawSheetOriginal;

    markFrameworkStep_(timing, "Raw source preserved for read  
without local rename: " + rawSheet.getName());

    const templateSheet = ss.getSheetByName(templateName);

    if (!templateSheet) {

        throw new Error(templateName + " is missing. Run Refresh  
Banner Template once before using Format Banner Report.");

    }

    markFrameworkStep_(timing, "Existing template verified: " +  
templateName);

    deleteSheetIfExists_(ss, outputName, rawSheet.getName(),  
templateName);
```

```
// --- ADVANCED API FAST CANVAS ---

// Right-size the grid based on the raw data size to avoid
dynamic resizing later

const requiredRows = Math.max(rawSheet.getLastRow() +
DATA_START_ROW, resolveTemplateRowCount_(sheetDef,
dashboard.globals));

const requiredCols = Math.max(templateSheet.getLastColumn(),
1);

const outputSheet = buildFastCanvasAdvanced_(ss, templateSheet,
outputName, requiredRows, requiredCols);

showSheetIfNeeded_(outputSheet, timing, "Banner output sheet
shown: " + outputName);

markFrameworkStep_(timing, "Advanced API structural canvas
built and formatted: " + outputName);

writeBannerReportDates_(outputSheet, reportDate);

markFrameworkStep_(timing, "Report title dates written");

copyRawBannerDataToOutput_(rawSheet, outputSheet);

markFrameworkStep_(timing, "Raw data bulk-copied to Advanced
API canvas");
```

```
    markFrameworkStep_(timing, "Production output validation  
skipped");
```

```
    archiveRawSourceAndDeleteLocal_(ss, rawSheet, rawName,  
outputSheet, timing);
```

```
    applyOutputVisibilityPolicy_(outputSheet, dashboard,  
sheetDef.sheetType, timing);
```

```
    refreshIndexAfterSheetWorkflow_("Format Banner Report");
```

```
    markFrameworkStep_(timing, "Banner formatter complete");
```

```
    return outputSheet;
```

```
});
```

```
}
```

```
function formatMonthlyBannerSheet_(rawSheet, parsedMonth, timing) {
```

```
    const ss = SpreadsheetApp.getActiveSpreadsheet();
```

```
    const dashboard = loadDashboardConfig_();
```

```
    const sheetDef = getSheetDefinitionByType_(dashboard,  
SHEET_TYPE.BANNER);
```

```
    const reportDate = parsedMonth.reportDate;
```

```
    const rawName = "Raw Data - Banners " + parsedMonth.monthLabel;
```

```
    const outputName = buildBannerReportOutputName_(reportDate);
```

```
    const templateName = "Template - Banner Report";
```

```
const templateSheet = ss.getSheetByName(templateName);

if (!templateSheet) throw new Error(templateName + " is missing.
Run Refresh Banner Template once before using Format Monthly
Sheets.");

deleteSheetIfExists_(ss, outputName, rawSheet.getName(),
templateName);

// --- ADVANCED API FAST CANVAS ---

const requiredRows = Math.max(rawSheet.getLastRow() +
DATA_START_ROW, resolveTemplateRowCount_(sheetDef,
dashboard.globals));

const requiredCols = Math.max(templateSheet.getLastColumn(), 1);

const outputSheet = buildFastCanvasAdvanced_(ss, templateSheet,
outputName, requiredRows, requiredCols);

showSheetIfNeeded_(outputSheet, timing, "Banner output sheet
shown: " + outputName);

writeBannerReportDates_(outputSheet, reportDate);

copyRawBannerDataToOutput_(rawSheet, outputSheet);

archiveRawSourceAndDeleteLocal_(ss, rawSheet, rawName,
outputSheet, timing);

applyOutputVisibilityPolicy_(outputSheet, dashboard,
sheetDef.sheetType, timing);
```

```
    return outputSheet;
}
```

The Disenrolled List append lag is a classic Google Sheets structural trap. When the script calls `exclusionSheet.insertRowsBefore(DATA_START_ROW, appendPayload.length)`, it forces Google's backend to recalculate the coordinates, formulas, and formatting for *every single row below it*. For a sheet with 700+ rows, doing this at the very top (row 5) is computationally brutal.

The solution is to **Append-to-Bottom, then Native Sort**. We will insert the new rows at the empty bottom of the sheet, bulk-apply the formatting, and then trigger a native Google Sheets sort (`range.sort()`). Native backend sorting takes ~0.5 seconds, completely eliminating the 25-second top-insertion penalty.

We will also implement Codex's recommendation for the **Index Signature Skip** to prevent the standalone `Create Disenrolled List` from wasting 10+ seconds rebuilding the Index if no sheets were actually created, deleted, or hidden.

Here is the implementation.

1. Refactor Disenrolled Append (Append-to-Bottom + Native Sort)

Replace your existing `appendDisenrolledDeltasToExclusionSheet_` function with this refactored version:

JavaScript

None

```
function appendDisenrolledDeltasToExclusionSheet_(exclusionSheet,
sourceHeaders, rowsToAppendByPMR, monthParts, timing) {

    const step = function(label) { if (timing)
markFrameworkStep_(timing, "Disenrolled move - exclusion append - "
+ label); };

    if (!exclusionSheet) return 0;

    updateDisenrolledExclusionReportDates_(exclusionSheet,
monthParts);
```

```
if (!rowsToAppendByPMR || rowsToAppendByPMR.size === 0) return 0;

const dashboard = loadDashboardConfig_();

const exclusionHeaders = getHeadersForSheetType_(dashboard,
SHEET_TYPE.DISENROLLED_EXCLUSION);

const sourceHeaderMap = buildHeaderIndexMap_(sourceHeaders ||
[]);

const sourcePMRIIdx = getPMRIndex_(sourceHeaderMap);

const exclusionHeaderMap =
buildHeaderIndexMap_(exclusionHeaders);

const exclusionPMRIIdx = getPMRIndex_(exclusionHeaderMap);

const exclusionDateIdx = findHeaderIndex_(exclusionHeaderMap,
["Disenrollment Effective Date", "Disenrollment Date", "Date of
Death"]);

const addedIdx =
exclusionHeaderMap[DISENROLLED_EXCLUSION_ADDED_HEADER];

const addedLabel = "Demo P " + (monthParts && monthParts.firstDay
? formatReportDateLabel_(monthParts.firstDay) : "");

const sourceIndexes = exclusionHeaders.map(function(header) {

    return Object.prototype.hasOwnProperty.call(sourceHeaderMap,
header) ? sourceHeaderMap[header] : -1;

});

const appendPayload = [];
```

```

rowsToAppendByPMR.forEach(function(rows) {
  (rows || []).forEach(function(sourceRow) {
    if (!sourceRow) return;

    const output = sourceIndexes.map(function(sourceIdx) {
      return sourceIdx === -1 ? "" : sourceRow[sourceIdx];
    });

    if (exclusionPMRIdx >= 0 && sourcePMRIdx >= 0 &&
!normalizePMR_(output[exclusionPMRIdx])) {
      output[exclusionPMRIdx] = sourceRow[sourcePMRIdx];
    }

    if (addedIdx !== undefined) output[addedIdx] = addedLabel;
    appendPayload.push(output);
  });
});

step("append payload built | Rows: " + appendPayload.length);

if (appendPayload.length === 0) return 0;

const width = Math.max(exclusionHeaders.length, 1);
if (exclusionSheet.getMaxColumns() < width) {

```

```

exclusionSheet.insertColumnsAfter(exclusionSheet.getMaxColumns(),
width - exclusionSheet.getMaxColumns());

}

// --- OPTIMIZATION: APPEND TO BOTTOM INSTEAD OF TOP-INSERTION
---

const currentLastRow = Math.max(exclusionSheet.getLastRow(),
DATA_START_ROW - 1);

const insertStartRow = currentLastRow + 1;

// Ensure we have physical rows at the bottom

const requiredMaxRows = insertStartRow + appendPayload.length -
1;

if (exclusionSheet.getMaxRows() < requiredMaxRows) {

    exclusionSheet.insertRowsAfter(exclusionSheet.getMaxRows(),
requiredMaxRows - exclusionSheet.getMaxRows());

}

step("row capacity ensured at bottom of Disenrolled Exclusion |
Rows: " + appendPayload.length);

const targetRange = exclusionSheet.getRange(insertStartRow, 1,
appendPayload.length, width);

```



```

    const writeResult =
writeDisenrolledAppendPayloadValues_(exclusionSheet,
insertStartRow, appendPayload, width);

    step("append payload values written | Rows: " +
appendPayload.length);

    try {

        if (currentLastRow >= DATA_START_ROW) {

            // Copy format cleanly from an existing data row instead of
relying on the row below

            exclusionSheet.getRange(DATA_START_ROW, 1, 1, width)

                .copyTo(targetRange,
SpreadsheetApp.CopyPasteType.PASTE_FORMAT, false);

            step("append payload formatting copied");

        }

    } catch (err) {

        logBestEffortWarning_("Incremental format brush extension
skipped: " + err.message);

    }

    // --- OPTIMIZATION: NATIVE SHEET SORT TO RESTORE
NEWEST-TO-OLDEST ---

    if (exclusionDateIdx !== -1) {

        const fullDataRange = exclusionSheet.getRange(DATA_START_ROW,
1, exclusionSheet.getLastRow() - DATA_START_ROW + 1, width);

```

```

        // Sort descending (false) on the disenrollment date column
        fullDataRange.sort({ column: exclusionDateIdx + 1, ascending:
false });

        step("native sheet sort applied to restore newest-to-oldest
order");
    }

    lockFinalOutputRowHeights_(exclusionSheet, "Disenrolled
Exclusion");

    step("Disenrolled Exclusion row heights locked");

    clearSheetRuntimeCachesForSheet_(exclusionSheet);

    return appendPayload.length;
}

```

2. Add Index Signature Short-Circuit

To prevent the `createIndexSheet()` function from wasting 10 seconds rebuilding the Index when the sheet structure hasn't actually changed, we will inject a signature check at the very top of the function.

Update your `createIndexSheet` function by **replacing the first few lines** with this code. Leave the rest of the function untouched:

JavaScript

```

None
function createIndexSheet() {

    const ss = SpreadsheetApp.getActiveSpreadsheet();

```

```

// --- OPTIMIZATION: INDEX SIGNATURE SHORT-CIRCUIT ---

// Generate a signature of current sheets, their IDs, and their
visibility states

const currentSignature = ss.getSheets().map(function(sh) {

    return sh.getSheetId() + ":" + sh.getName() + ":" + (typeof
sh.isSheetHidden === "function" ? sh.isSheetHidden() : false);

}).join("|");

const props = PropertiesService.getDocumentProperties();

const lastSignature =
props.getProperty("RFF_INDEX_SHEET_SIGNATURE");

let sheet = ss.getSheetByName(INDEX_SHEET);

// If the workbook architecture hasn't changed, immediately
return to bypass the 10-second rebuild

if (sheet && lastSignature === currentSignature) {

    return sheet;

}

// -----

const dashboard = loadDashboardConfig_();

const globals = dashboard.globals || {};

```

```

const theme = getThemeColorsFromBase_("#668BCC", globals);

const indexMonthParts = getMonthDateParts_(new Date());

if (!sheet) {
    sheet = ss.insertSheet(INDEX_SHEET, 0);
} else {
    try {

sheet.getDataRange().clearContent().clearFormat().breakApart();

        } catch (err) {

            logBestEffortWarning_("Index governed-range reset skipped: "
+ err.message);

        }
    }

    // ... (Keep the rest of your existing createIndexSheet() logic
    exactly as is) ...

```

Here is the implementation to resolve Priority 4.

The core issue with **Hide Monthly Import Sheets** taking 43+ seconds is that it blindly triggers **enforceGlobalSheetSortOrder_** at the end of its run, regardless of whether any sheets actually changed visibility. **enforceGlobalSheetSortOrder_** is an extremely heavy operation that reads every single tab in the workbook, calculates their global rank, and checks their visibility states.

By adding a `stateChanged` flag and tracking `alreadyHidden` sheets, we can completely short-circuit the global sort and index rebuild if the sheets were already hidden. We also add the granular telemetry Codex recommended to break down that broad "Complete" timing step.

Hide Monthly Sheets Refactor (Short-Circuit & Granular Telemetry)

Replace your existing `hideMonthlySheetsBySpecs_` function with this updated version:

JavaScript

None

```
function hideMonthlySheetsBySpecs_(processName, parsedMonth, specs)
{
    return runFrameworkTimed_(processName + " " +
        parsedMonth.monthLabel, function(timing) {

        const ss = SpreadsheetApp.getActiveSpreadsheet();

        const result = { hidden: [], skipped: [], alreadyHidden: [] };

        let stateChanged = false;

        markFrameworkStep_(timing, "Candidate lookup started");

        specs.forEach(function(spec) {

            const sheets = findArchiveMonthlyCandidateSheets_(ss,
                spec.keywords, parsedMonth.monthLabel);

            if (!sheets.length) {

                result.skipped.push(spec.label + " not found");

                markFrameworkStep_(timing, "Lookup skipped: " + spec.label
                    + " not found");

                return;
            }
        });
    });
}
```

```
}

sheets.forEach(function(sheet) {
  const sheetName = sheet.getName();
  try {
    if (!sheet.isSheetHidden()) {
      sheet.hideSheet();
      result.hidden.push(sheetName);
      stateChanged = true;
      markFrameworkStep_(timing, "Sheet hidden: " +
sheetName);
    } else {
      result.alreadyHidden.push(sheetName);
      markFrameworkStep_(timing, "Sheet already hidden
(skipped): " + sheetName);
    }
  } catch (err) {
    result.skipped.push(sheetName + ": " + err.message);
    markFrameworkStep_(timing, "Error hiding sheet: " +
sheetName, err.message);
  }
});
});
```

```

        // --- OPTIMIZATION: SHORT-CIRCUIT GLOBAL SORT IF NOTHING
        CHANGED ---

        if (stateChanged) {

            markFrameworkStep_(timing, "Visibility state changed -
            running global sort");

            enforceGlobalSheetSortOrder_(parsedMonth.monthParts);

            markFrameworkStep_(timing, "Global sort complete");

        } else {

            markFrameworkStep_(timing, "Visibility state unchanged -
            global sort and index refresh skipped");

        }

        notify_(processName + " complete for " + parsedMonth.monthLabel
        + ".\n" +

            "Hidden: " + result.hidden.length + "\n" +

            "Already Hidden: " + result.alreadyHidden.length + "\n"
        +

            "Skipped: " + result.skipped.length);

        return result;

    });
}

```

Why this works:

1. **Granular Telemetry:** Instead of a single 43-second block, your Framework Timing Report will now log exactly how long it takes to find the sheets, evaluate each specific sheet's visibility, and hide it.
2. **Idempotent Speed:** If a user accidentally clicks "Hide Monthly Import Sheets" twice, the second run will detect that `sheet.isSheetHidden()` is `true`, flag it as `alreadyHidden`, and skip the heavy `enforceGlobalSheetSortOrder_` engine entirely. The second run will take less than 1 second.